Structs and Arrays

Where We Left Off

A variable is a **symbolic name** *for an address*

The **contents** *of that address is the value*

We used atomic types:

i32, u8, f64, bool, char

int32_t, uint8_t, double, bool, char

One name, one value, one location

Real Data Comes in Groups

A single pressure reading is really **three** *numbers*

the raw ADC count from the sensor
the pressure in psi we computed
the time the sample was taken

They are three variables that belong together

The language should let allow to group them efficiently

Loose Variables Do Not Scale

```
uint32_t   adc_counts_1 ;
float      psi_1 ;
uint64_t   timestamp_1 ;

uint32_t   adc_counts_2 ;
float      psi_2 ;
uint64_t   timestamp_2 ;
```

Nothing ties them together
The compiler cannot help you keep them straight

The Struct

A **struct** *groups variables into a* **new type <T>**

The grouped variables are called **members** *or* **fields**

They are stored in one **consecutive block** *of memory*

One address locates the whole group

Same idea in C, C++, and Rust

Also known as a record

Defining a struct in C

```
struct sample {
    uint32_t    adc_counts ;
    float       psi ;
    uint64_t    timestamp_ms ;
} ;
```

This defines a **type**, *not a variable*

No memory is allocated yet

Declaring the variable in C

struct *sample my_sample ;*

The keyword struct is **required** *in C*

This is an instance of type **struct** *sample*

<T> is <struct sample>

struct sample sample2 ;

sample2 is a second instance of the same new type

Typedefs

```
struct sample {
    uint32_t    adc_counts ;
    float       psi ;
    uint64_t    timestamp_ms ;
} ;

typedef struct sample sample_t ;
sample_t my_sample ;
sample_t sample2 ;
```

Now memory exists: sizeof(struct sample) bytes

Better typedef

Even better: use **typedef** *with the* **struct** *definition:*

```
typedef struct sample {
    uint32_t    adc_counts ;
    float       psi ;
    uint64_t    timestamp_ms ;
} sample_t ;
```

Variable definitions again are:

```
sample_t my_sample ;
sample_t sample2 ;
```

Now memory exists: sizeof(sample) bytes

Defining a struct in C++

Same syntax as C:

struct sample { ... } ;

~~struct~~ sample my_sample ;

The struct keyword is **not required** *when declaring*

C++ also has **class**

struct members are **public** *by default*

class members are **private** *by default*

Otherwise functionally equivalent

Defining a struct in Rust

```
struct Sample {
        adc_counts : u32,
        psi : f32,
        timestamp_ms : u64,
}
```

Fields are separated by **commas**
No semicolon after the closing brace
Type names are **CamelCase**, *fields are* **snake_case**

Programming Case Styles

Case Style	*Format /Rule*	*Example*	*Common Use*
Camel Case *(camelCase)*	*First letter lowercase, next words capitalized, no spaces*	**userId**	*Variables and functions in JavaScript, Java, C#*
Upper Camel Case *(UpperCamelCase)*	*Every word capitalized no spaces*	**UserId**	*Types in Rust*
Pascal Case *(PascalCase)*	*Every word starts with a capital letter, no spaces*	**UserId**	*Classes and types in C#, Java, TypeScript*
Snake Case *(snake_case)*	*All lowercase, words separated by underscores (_)*	**user_id**	*Variables and functions in Python, Ruby*
Screaming Snake *(SCREAMING_SNAKE)*	*All uppercase, words separated by underscores (_)*	**USER_ID**	*Constants in Python, Java, macros C, C++*
Kebab Case *(kebab-case)*	*All lowercase, words separated by hyphens (-)*	**user-id**	*URLs, CSS classes, HTML elements*

Defining a struct in Rust

struct *Sample {*
adc_counts : u32,
psi : f32,
timestamp_ms : u64,
}
Fields are separated by **commas**
No semicolon after the closing brace
Type names are **CamelCase**, *fields are* **snake_case**

You can turn if off for a specific struct by using
#[allow(non_camel_case_types)]

Not recommended to turn off the standard
Makes it easier to read others code

Rust Programming Guidelines

In general, Rust tends to use UpperCamelCase for "type-level" constructs (types and traits) and snake_case for "value-level" constructs. More precisely:

Item	Convention
Crates	unclear
Modules	snake_case
Types	UpperCamelCase
Traits	UpperCamelCase
Enum variants	UpperCamelCase
Functions	snake_case
Methods	snake_case
General constructors	new or with_more_details
Conversion constructors	from_some_other_type
Macros	snake_case!
Local variables	snake_case
Statics	SCREAMING_SNAKE_CASE
Constants	SCREAMING_SNAKE_CASE
Type parameters	concise UpperCamelCase, usually single uppercase letter: T
Lifetimes	short lowercase, usually a single letter: 'a, 'de, 'src
Features	unclear but see C-FEATURE

Giving it a value: C and C++

struct sample my_sample = { 1456, 50.0f, 12345 } ;

Order matters! Better, name the members:

```
struct sample my_sample = {
        .adc_counts         = 1456,
        .psi                = 50.0f,
        .timestamp_ms       = 12345
} ;
```

But we still need this definition for the type to precede it:

```
typedef struct sample {
        uint32_t            adc_counts ;
        float               psi ;
        uint64_t            timestamp_ms ;
} sample_t ;
```

Giving it a value: Rust

```
let my_sample = Sample {
    adc_counts      : 1456,
    psi             : 50.0,
    timestamp_ms    : 12345,
} ;
```

Every *field must be given a value*
The compiler will not let you forget one
Use **let mut** *if you intend to change it later*

Reaching a Member: the dot

C and C++:

```
my_sample.psi = 50.0f ;
printf("%f\n", my_sample.psi) ;
```

Rust:

```
my_sample.psi = 50.0 ;
println!("{}", my_sample.psi) ;
```

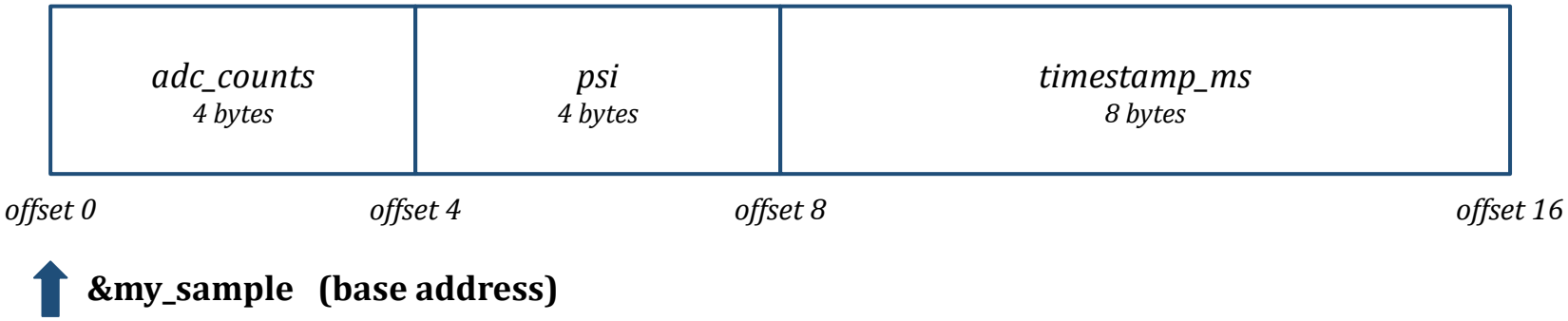
The **dot operator** *is the same in all three*

A struct in Memory

The struct has **one base address** *: &my_sample*

Each member sits at a fixed **offset** *from that base*

address of member = base address + offset



Size and Offset

The compiler knows every offset at **compile time**

C and C++:

sizeof(struct sample)
#include <stddef.h>
offsetof(struct sample, psi)

Rust:

std::mem::size_of::<Sample>()

Print these in your own program - do not guess

Padding and Alignment

Hardware reads memory fastest on **natural boundaries**

a 4 byte value wants an address divisible by 4
an 8 byte value wants an address divisible by 8

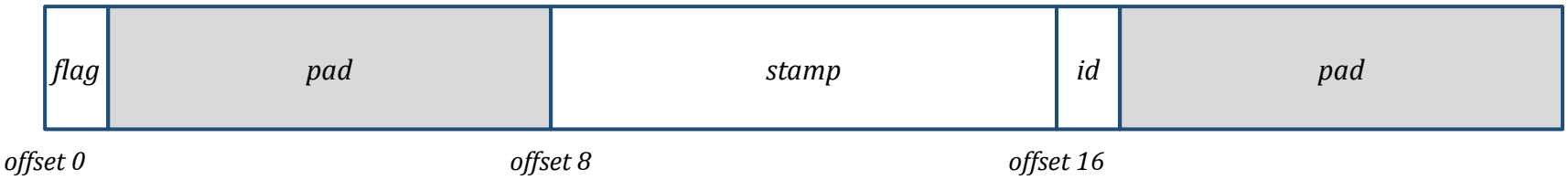
The compiler inserts **padding bytes** *to make this true*

So sizeof(struct) can be **larger** *than the sum of its parts*

Let's look at mempractice again

Padding Costs Real Memory

```
struct bad {
        uint8_t         flag ;
        uint64_t        stamp ;
        uint8_t         id ;
} ;
```



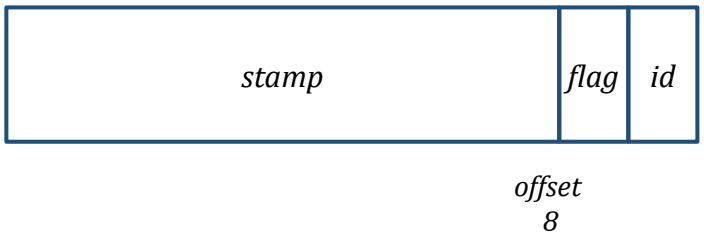
sizeof is 24 bytes: 10 of data, 14 of padding

Order Your Members

Rule of thumb: declare members **largest to smallest**

```
struct good {
        uint64_t        stamp ;
        uint8_t         flag ;
        uint8_t         id ;
} ;
```

sizeof is now **16** *bytes, not 24*



Controlling the Layout

C and C++ (compiler specific):

```
#pragma pack(1)
__attribute__((packed))
```

Rust:

```
#[repr(C)]
struct Sample { ... }
```

Rust may **reorder** *fields unless you say #[repr(C)]*

Pointers to structs

C and C++:

*struct sample *p = &my_sample ;*

p->psi = 50.0f ; *same as (*p).psi*

Rust:

let p = &my_sample ;

p.psi *Rust dereferences for you*

There is **no -> operator** *in Rust*

Pointers

Pointers are variables that are interpreted as addresses

It is an indirection to the memory of interest

We use the notation (variable) to denote the contents of an address

A pointer is the address of the contents

pointer to a variable <= & (variable)

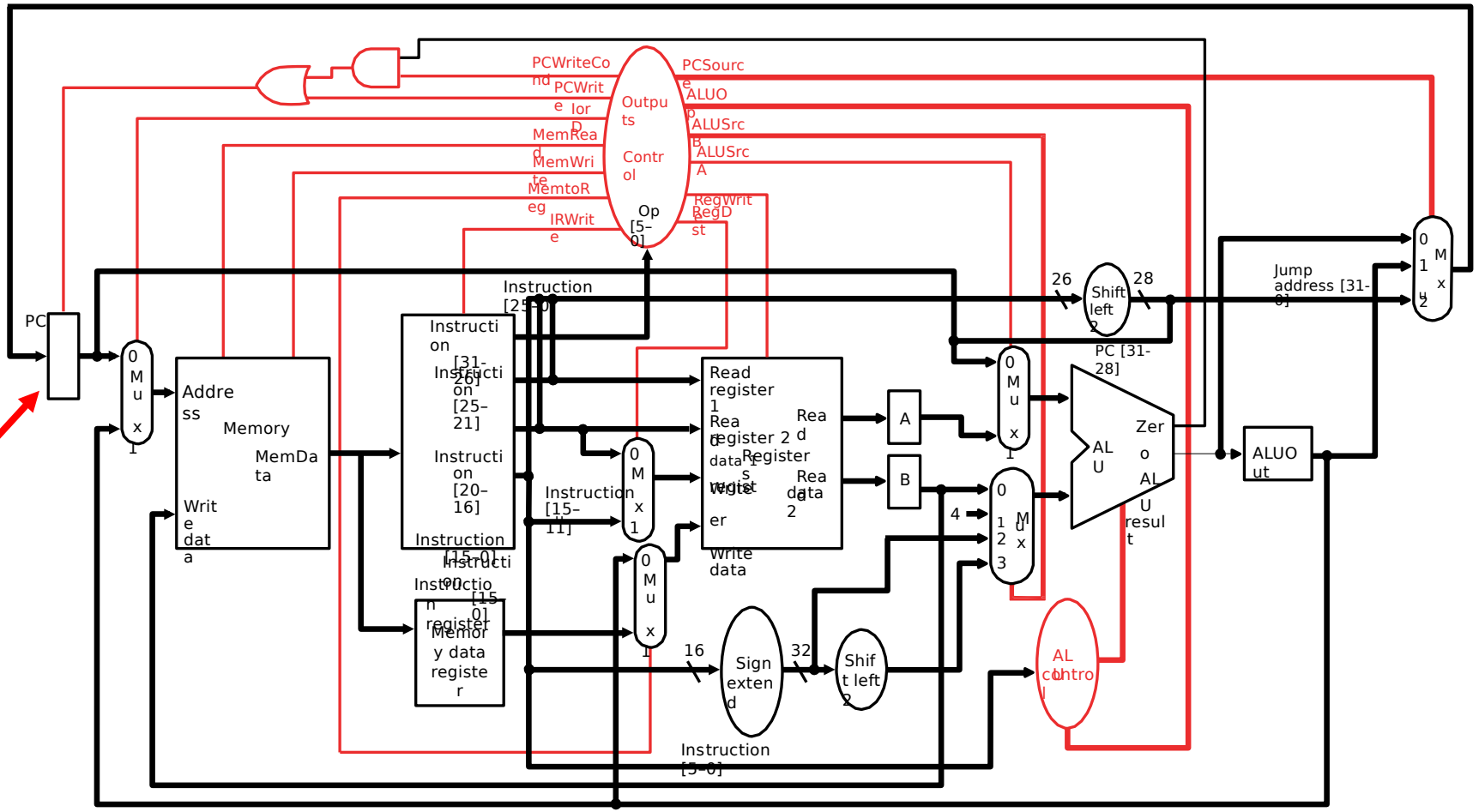
If we wish to get the contents pointed by the pointer, we dereference the pointer

*We dereference using the * operator*

contents_pointed_to <= ((pointer))

Hardware Pointers

PC:
Program
Counter



The PC or Program Counter is an address. It is the address of a register holding the address of the next instruction to be executed.

The program counter is the hardware equivalent of a pointer

Handles

A handle is a pointer to a pointer

Handle <= & & (variable)

To get the contents pointed by the handle, we need to dereference twice: handle -> pointer -> contents

*We dereference using the ** operator (twice)*

contents <= (((handle)))

See the mempractice tool in pointer mode

Structs inside structs

```
struct reading {
        struct sample       sample ;
        uint8_t             sensor_id ;
} ;
```

Reach in with more dots:

```
my_reading.sample.psi
```

Still one block of memory, still fixed offsets

Build complex objects through hierarchy

Passing a struct to a function

By value copies **every byte***:*

void show(struct sample s) ;

Pass a pointer or reference instead:

*void show(const struct sample *s) ;*

void show(const sample &s) ; *C++*

fn show(s : &Sample) { ... } *Rust*

In Rust, passing by value **moves** *ownership*

Many of the Same Thing

One sample is not enough

We want the last **100** *samples*

Copy and paste 100 variable definitions?

my_sample_1, my_sample_2, ... my_sample_100

There must be a better way

The Array

An **array** *is N variables of the* **same** *type*

Stored **consecutively** *in memory*

One name, one base address

Individual elements selected by an **index**

The index starts at **0**

Declaring arrays in C and C++

```
uint32_t    counts[100] ;
float       psi[100] = { 0.0f } ;
```

The length is part of the declaration

C++ also offers a safer array:

```
#include <array>
std::array<uint32_t,100> counts ;
```

Declaring arrays in Rust

let counts : [u32 ; 100] = [0 ; 100] ;

let mut psi : [f32 ; 100] = [0.0 ; 100] ;

The type is [type ; length]

The **length is part of the type**

[0 ; 100] means "the value 0, one hundred times"

Indexing starts at zero

counts[0] *the* **first** *element*
counts[99] *the* **last** *element*
counts[100] *does* **not** *exist*

Why zero?

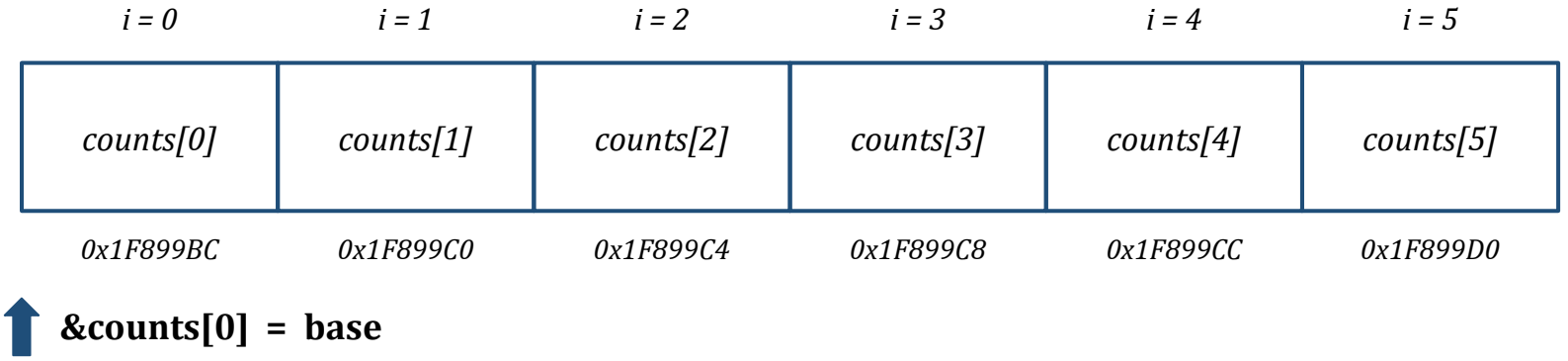
Because the index is an **offset** *from the base address*

Element 0 is 0 elements away from the start

An array in memory

*address of element i = base + i * sizeof(element)*

For uint32_t, every element is 4 bytes apart



&counts[0] = base

*&counts[3] = base + 3 * 4 = 0x1F899C8*

How many elements?

C:

sizeof(counts) / sizeof(counts[0])

C++ (std::array or std::vector):

counts.size()

Rust:

counts.len()

Out of bounds

C and C++: **no checking**

counts[100] reads or writes memory that is not yours

Undefined behavior - may crash, may silently corrupt

Rust: **checked at run time**

the program panics with a clear message and stops

This single difference is a large part of why Rust exists

Walking an array

C and C++:

```
for (int i = 0 ; i < 100 ; i++) {
        psi[i] = adc_to_psi(counts[i]) ;
}
```

Rust:

```
for i in 0..100 {
        psi[i] = adc_to_psi(counts[i]) ;
}
```

Arrays of structs

Put the two ideas together:

C, C++: *struct sample history[100] ;*
history[7].psi = 50.0f ;

Rust: *let mut history : [Sample ; 100] ;*
history[7].psi = 50.0 ;

Elements are sizeof(struct sample) bytes apart

Two dimensions

uint8_t image[3][4] ; *3 rows of 4*

let image : [[u8 ; 4] ; 3] ; *Rust*

*Stored one row after another (***row major***)*

*image[1][2] is at base + (1*4 + 2) bytes*

It is still one flat block of memory

When the size is not known yet

C: *malloc / free (later lecture)*

C++:
```
#include <vector>
std::vector<uint32_t> counts ;
counts.push_back(1456) ;
```

Rust:
```
let mut counts : Vec<u32> = Vec::new() ;
counts.push(1456) ;
```

Now Let Us Look Inside

You can now describe **structures** *of memory*

How do you **see** *them while the program runs?*

printf tells you what you decided to ask about
before *you knew what was wrong*

A **debugger** *lets you stop and look at anything*

What a debugger does

Loads your program but keeps **control** *of it*

*Stops it where you choose (***breakpoints***)*

Reads the memory of the stopped program

Uses the **symbol table** *to turn addresses back into names*

Shows a variable as its **type***, not as raw bytes*

Debug information

The symbol table and type descriptions are placed in the executable by the compiler. The format is called **DWARF**

You have to ask for it:

gcc -g -O0 main.c	*-o main*
g++ -g -O0 main.cpp	*-o main*
cargo build	*(dev profile has -g)*

-O0 *turns off optimization so the code matches your source*

The tools

gdb - *GNU debugger, Linux and embedded targets*

lldb - *LLVM debugger, the default on macOS*

rust-gdb / rust-lldb - *add Rust pretty printers*

VS Code - *a graphical front end for all of them*

All of them do the same job with different spelling

Starting a session

gcc -g -O0 main.c -o main

gdb ./main

(gdb) break main
(gdb) run

The program stops at the first line of main
and hands control back to you

Everyday gdb Commands

Controlling the program:

break <file:line>	*set a breakpoint*
run	*start the program*
next	*run one line, step* **over** *calls*
step	*run one line, step* **into** *calls*
continue	*run to the next breakpoint*
finish	*run until this function returns*

Everyday gdb Commands

Looking around:

print <expr>	*show a value*
info locals	*show every local variable*
backtrace	*show the call stack*
list	*show the source*
quit	*leave the debugger*

Short forms: b r n s c bt p l q

Looking at a variable

(gdb) **print** *my_integer*
$1 = 55

(gdb) **print/x** *my_integer*
$2 = 0x37

(gdb) **print** *&my_integer*
*$3 = (int64_t *) 0x7ffd1f899bc*

Contents and address - the two ideas from lecture one

Looking at a struct

(gdb) **print** *my_sample*
$1 = {adc_counts = 1456, psi = 50, timestamp_ms = 12345}

(gdb) **print** *my_sample.psi*
$2 = 50

One command prints the **whole structure***, with names*

What type is it?

(gdb) **ptype** *my_sample*

```
type = struct sample {
    uint32_t adc_counts;
    float psi;
    uint64_t timestamp_ms;
}
```

The debugger knows this from the **DWARF** *data*

Looking at an array

(gdb) **print** *counts*
$1 = {1456, 1442, 1460, 0 <repeats 97 times>}

(gdb) **print** *counts[7]*
$2 = 0

One command prints **every element**

Part of an array: the @ operator

(gdb) **print** *counts[2]@4*
$3 = {1460, 0, 0, 0}

Four elements, starting at index 2

Most useful when you have a pointer and a length:

*(gdb) print *ptr@10*

Looking at raw memory

(gdb) **x/16xb** *&my_sample*

x	*examine*
16	*how many*
x	*format: hex (d, u, c, s, i)*
b	*size: byte (h, w, g)*

This is where you can **see the padding bytes**

Who changed my variable?

(gdb) **watch** *psi[3]*
(gdb) continue

Hardware watchpoint 2: psi[3]
Old value = 0
New value = 50
update_sample () at main.c:42

The hardest bug to find by reading code

Debugging Rust

cargo build
rust-gdb ./target/debug/myprog
rust-lldb ./target/debug/myprog

The wrappers load **pretty printers***, so a Vec or a String prints readably instead of as internal pointers*

Rust symbol names include the crate:
(gdb) break myprog::main

The same ideas in lldb

lldb ./main

(lldb) breakpoint set --name main *br s -n main*
(lldb) run
(lldb) next / step / continue
(lldb) frame variable *fr v*
(lldb) print my_sample
(lldb) memory read -f x -s 1 -c 16 &my_sample

Different spelling, identical ideas

The debugger in VS Code

Install the **C/C++** *or* **CodeLLDB** *extension*

For Rust: **rust-analyzer** *plus CodeLLDB*

Click in the gutter to set a breakpoint

F5 *start,* **F10** *step over,* **F11** *step into*

The VARIABLES pane

Lists every local variable, updated as you step

Expand a **struct** *to see its members*

Expand an **array** *to see its elements*

Hover over a name in the source to see its value

It is gdb or lldb underneath - the ideas still apply

A method for using it

1. *Reproduce the failure*
2. *Set a breakpoint* **before** *the suspect line*
3. *Look at the variables - are they what you expect?*
4. *Step one line at a time and watch them change*
5. *The first value that* **surprises** *you is near the bug*

What we can now describe

Variable	*one name, one value, one address*
Struct	*different types, one block, fixed offsets*
Array	*one type repeated, base + index * size*
Debugger	*stop the program and look at all of it*

Every one of these is still just **memory**

Let's use the debugger

Exercise: Using **structpractice**